

AI-Powered Observability & Telemetry Platform

A Complete, Beginner-Friendly Walkthrough of the Whole System

Java 21 · Spring Boot

gRPC

Apache Kafka

PostgreSQL

Redis

Python · FastAPI

PyTorch · scikit-learn

React

Docker Compose

Complete architecture, end-to-end data flow, code explanations,
design rationale, and interview preparation Q&A.

Read this top-to-bottom and you will understand the entire project.

Table of Contents

1. The 30-Second Summary — What Is This Project?
2. The Big Picture — Architecture at a Glance
3. The Complete Data Flow — Journey of One Metric
4. Technology Stack & Why Each Was Chosen
5. Service Deep-Dive #1 — Telemetry Collector (gRPC)
6. The Message Bus — Apache Kafka & Its Topics
7. Service Deep-Dive #2 — Processing Service
8. The Data Stores — PostgreSQL & Redis
9. Service Deep-Dive #3 — AI Anomaly Detection (the brain)
10. Service Deep-Dive #4 — Alert Service
11. Service Deep-Dive #5 — Backend API + WebSocket
12. Service Deep-Dive #6 — React Dashboard
13. Two Complete End-to-End Walkthroughs
14. Cross-Cutting Design Concepts (the "why it's good")
15. Interview Q&A — Anticipated Questions & Strong Answers
16. Glossary & Quick Reference

1. The 30-Second Summary — What Is This Project?

Imagine you run dozens of software services (an API gateway, an order service, a payment service, etc.). Each one constantly produces **numbers about its own health** — CPU usage, memory, request latency, error rate, and so on. These numbers are called **metrics** or **telemetry**.

This project is a **platform that collects all those numbers, stores them, uses Artificial Intelligence to automatically spot when something looks abnormal (an "anomaly"), figures out the likely cause, sends alerts to humans (Slack / email), and shows everything live on a web dashboard.**

In one line: It is a mini version of commercial tools like *Datadog*, *New Relic*, or *Dynatrace* — but built from scratch with open-source pieces, and with its own AI engine for detecting problems.

What makes it "AI-Powered"?

Instead of a human writing fixed rules like "alert me if CPU > 90%", the platform runs **three machine-learning models** that *learn what normal looks like* for each service and each metric, and then flag values that deviate from that learned normal. It also has a **Root Cause Analyzer** that suggests *why* the problem might be happening.

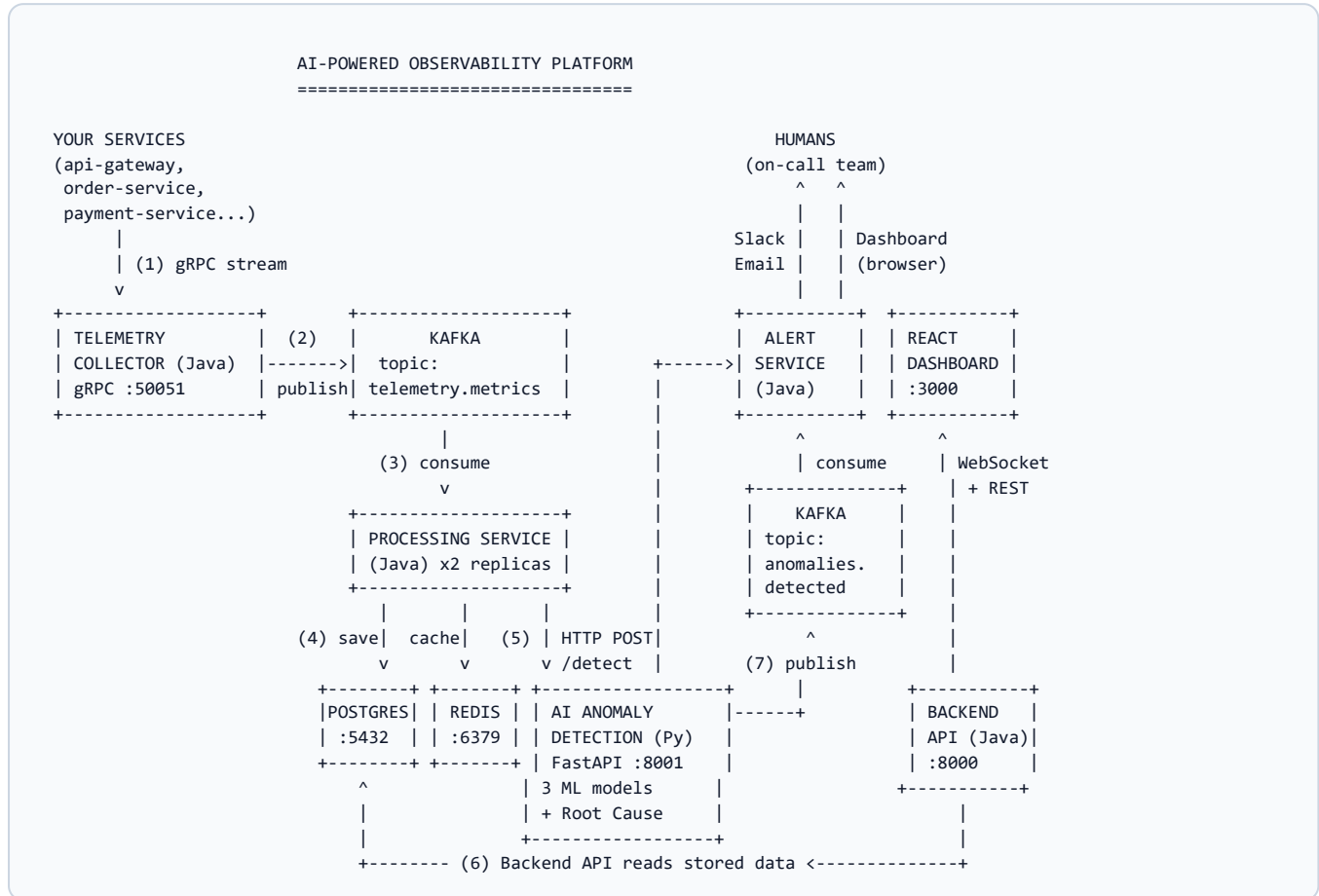
The seven moving parts

#	Component	Job (in plain English)	Tech
1	Telemetry Collector	The front door. Receives metrics from services over a fast network protocol.	Java + gRPC
2	Kafka	The conveyor belt. Buffers messages so producers and consumers don't block each other.	Apache Kafka
3	Processing Service	The worker. Validates metrics, saves them to the database + cache, and asks the AI to check them.	Java + Spring
4	AI Anomaly Detection	The brain. Runs 3 ML models to score how abnormal each metric is, then explains the cause.	Python + PyTorch + scikit-learn
5	Alert Service	The messenger. Turns detected anomalies into Slack / email alerts (with anti-spam).	Java + Spring
6	Backend API	The librarian. Serves stored data to the UI via REST + pushes live updates via WebSocket.	Java + Spring
7	React Dashboard	The screen. Live charts, service health, and an anomaly feed for humans to watch.	React + Recharts

Plus two data stores — **PostgreSQL** (permanent storage) and **Redis** (fast temporary cache) — and everything is packaged with **Docker Compose** so the whole thing starts with one command.

2. The Big Picture — Architecture at a Glance

Here is the entire system on one page. Follow the arrows from left to right: data is born on the left (your services) and ends up on the right (a human looking at a dashboard or getting a Slack message).



The numbered steps (the heartbeat of the system)

1. **Send:** A service streams metric batches to the Collector over **gRPC** (a fast binary protocol).
2. **Publish:** The Collector converts each batch to JSON and publishes it to the Kafka topic `telemetry.metrics`.
3. **Consume:** The Processing Service reads messages off Kafka.
4. **Store:** It validates each metric, bulk-inserts into PostgreSQL, and updates Redis (latest value + a sliding window of recent values).
5. **Detect:** It makes an async HTTP call to the Python AI service, sending the recent window of values per metric.
6. **Score & explain:** The AI runs 3 models, computes an ensemble anomaly score, classifies severity, and (for serious ones) generates root-cause text.
7. **Fan-out:** If anomalies are found, the AI publishes them to Kafka topic `anomalies.detected`. Two services consume that topic: the **Alert Service** (sends Slack/email) and the **Backend API** (saves to DB + pushes to the dashboard live via WebSocket).

Key idea: The services are *decoupled*. They never call each other directly in the hot path — they talk through Kafka. If the AI service is slow or down, metrics keep flowing and getting stored; detection simply catches up later. This is what makes the system resilient.

3. The Complete Data Flow — Journey of One Metric

Let's follow a single number — say `cpu_usage = 95.0` from the `payment-service` — on its entire trip through the platform. This is the most important section: if you understand this journey, you understand the project.

```
STEP 0 A service measures itself: cpu_usage=95, latency_p99=800, error_rate=10 ...
      It packs ~9 metrics into a "MetricBatch" and streams it over gRPC.

STEP 1 TELEMETRY COLLECTOR (TelemetryCollectorService.streamMetrics)
      - validates: serviceId not blank, batch not empty
      - serviceRegistry.touch() -> remembers "this service was seen just now"
      - converts protobuf -> JSON map
      - kafkaPublisher.publish(serviceId, json) [serviceId = Kafka KEY]
      - replies with an Ack {success=true, stored=9}

STEP 2 KAFKA topic "telemetry.metrics" (12 partitions)
      - message lands on a partition decided by hash(serviceId)
      - so ALL messages from payment-service go to the SAME partition -> ordered

STEP 3 PROCESSING SERVICE (MetricKafkaConsumer.consume) [3 threads, 2 replicas]
      - deserializes JSON -> MetricBatchPayload
      - processor.process(batch)
      - ONLY commits the Kafka offset AFTER success (at-least-once delivery)

STEP 4 MetricProcessor.process()
      a) upsert the service row in PostgreSQL (create if first time)
      b) for each metric: check it's in METRIC_SCHEMA, clamp to valid range
      c) metricRepo.saveAll(...) -> ONE bulk insert for the whole batch
      d) Redis: store latest value (HASH) + push to sliding window (LIST, last 60)
      e) fire async HTTP POST to AI: { metric_windows: {cpu_usage:[...60 values...]} }

STEP 5 AI ANOMALY DETECTION (POST /detect)
      for each metric window (>= 5 points):
        run 3 models in parallel:
          Isolation Forest -> if_score
          LSTM Autoencoder -> lstm_score
          Statistical base -> base_score
        ensemble = 0.40*if + 0.40*lstm + 0.20*base
        if any model fired AND ensemble high -> it's an anomaly
        classify severity (low/medium/high/critical)
        if ensemble >= 0.75 -> RootCauseAnalyzer.analyze() adds an explanation

STEP 6 AI publishes anomaly to KAFKA topic "anomalies.detected"

STEP 7a ALERT SERVICE consumes it
      severity filter -> dedup (Redis) -> format -> save to DB -> Slack (+ email if high/critical)

STEP 7b BACKEND API consumes the SAME message
      saves anomaly to PostgreSQL (so REST API can query history)
      a @Scheduled job pushes a fresh snapshot to the browser every 1 second via WebSocket

STEP 8 REACT DASHBOARD receives the WebSocket snapshot
      updates the live charts, the service health badges, and the Anomaly Feed
      the on-call engineer sees the red "critical" card light up.
```

Total latency: From a metric arriving to it being stored is a few milliseconds. From an anomaly happening to a Slack alert is typically **30–60 seconds** — because the AI needs a *window* of recent data before it can judge whether the latest value is abnormal.

4. Technology Stack & Why Each Was Chosen

Layer	Technology	Why this one (the reasoning)
Ingestion protocol	gRPC (with bidirectional streaming)	Binary & compact (uses Protocol Buffers), much faster than JSON/REST for high-volume metric streaming. Streaming lets a service keep one connection open and fire metrics continuously instead of one HTTP request per batch.
Message bus	Apache Kafka	Decouples producers from consumers, buffers huge bursts, keeps data durably for days, and lets multiple independent consumers read the same stream (Alert Service + Backend API both read anomalies). Partitions give horizontal scaling + ordering per key.
App services	Java 21 + Spring Boot	Strong typing, mature ecosystem, excellent Kafka/JPA/WebSocket support, great for high-throughput backend work. Java 21 gives modern features (records, virtual-thread-ready, pattern matching).
AI / ML	Python + FastAPI + PyTorch + scikit-learn	The entire ML world is Python-native. Rather than fight to do ML in Java, the design draws a clean language boundary: Java handles data plumbing, Python handles the math, and they talk over simple HTTP.
Permanent store	PostgreSQL 15 (time-partitioned)	Reliable, SQL-queryable, supports table partitioning by day so old metric data can be dropped cheaply. Good for time-series at moderate scale.
Fast cache	Redis 7	In-memory, sub-millisecond. Used for "latest value" lookups, the rolling window the AI reads, and atomic alert deduplication locks.
Dashboard	React 18 + Recharts	Component-based UI, easy live updates from WebSocket state, Recharts for clean metric charts.
Packaging	Docker Compose	One file describes all 10+ containers, their config, dependencies, and health checks. docker compose up starts the whole platform reproducibly.

The headline architectural decision — "polyglot microservices": use the *best language for each job* (Java for robust concurrent data services, Python for ML) and glue them together with language-neutral protocols (gRPC, Kafka, HTTP/JSON). This is a very common pattern in real companies.

Ports cheat-sheet

Service	Port	Protocol
Telemetry Collector	50051 (gRPC), 8080 (health)	gRPC / HTTP
Processing Service	8082	HTTP (health only)
Alert Service	8083	HTTP (health only)
Backend API	8000	REST + WebSocket
AI Anomaly Detection	8001	HTTP (FastAPI)
Dashboard	3000 → 80 (nginx)	HTTP
PostgreSQL / Redis / Kafka	5432 / 6379 / 29092	TCP

5. Service Deep-Dive #1 — Telemetry Collector (gRPC)

Java 21 · Spring Boot · net.devh gRPC · Kafka producer

Job: Be the front door. Accept metric streams from any number of services as fast as possible, do minimal validation, and hand them to Kafka. It deliberately does *not* touch the database — it must stay light and fast.

The contract: `telemetry.proto`

gRPC services are defined in a `.proto` file. This is the shared "contract" — both the Java server and the Python test client generate code from it, so they always agree on the message shapes. It defines 4 RPCs:

```
service TelemetryService {
  rpc StreamMetrics      (stream MetricBatch) returns (stream Ack);    // hot path
  rpc RegisterService   (ServiceInfo)        returns (RegistrationResponse);
  rpc GetServiceHealth  (ServiceQuery)        returns (HealthResponse);
  rpc SubscribeAlerts   (AlertSubscription)    returns (stream AlertEvent);
}
```

The important one is `StreamMetrics`: it is **bidirectional streaming** — the client keeps sending `MetricBatch` messages and the server keeps replying with `Ack` messages, all over a single long-lived connection.

Key code — handling the stream

```
@GrpcService
public class TelemetryCollectorService extends TelemetryServiceGrpc.TelemetryServiceImplBase {

  public StreamObserver<MetricBatch> streamMetrics(StreamObserver<Ack> responseObserver) {
    return new StreamObserver<>() {
      public void onNext(MetricBatch batch) {
        // called per incoming batch
        if (batch.getServiceId().isBlank() || batch.getMetricsCount() == 0) {
          responseObserver.onNext(buildAck(false, "validation failed", 0));
          return;
        }
        // drop bad batches
      }
      serviceRegistry.touch(batch.getServiceId(), ...); // update "last seen"
      String json = objectMapper.writeValueAsString(batchToMap(batch));
      kafkaPublisher.publish(batch.getServiceId(), json); // serviceId = Kafka key
      responseObserver.onNext(buildAck(true, "ok", batch.getMetricsCount()));
    };
    public void onError(Throwable t) { /* client disconnected */ }
    public void onCompleted() { responseObserver.onCompleted(); }
  };
}
```

Why serviceId is used as the Kafka key: Kafka sends all messages with the same key to the same partition. Using `serviceId` as the key guarantees that all metrics from one service are processed *in order* — which matters for time-series correctness.

The Kafka producer tuning (CollectorConfig)

Setting	Value	What it does
<code>acks=all</code>	all	Wait for replicas to confirm → no data loss.
<code>enable.idempotence</code>	true	Exactly-once writes per partition (no accidental duplicates from retries).
<code>linger.ms</code>	5	Wait 5ms to batch more messages together → fewer, bigger sends.
<code>batch.size</code>	65536	64 KB batches before flushing.
<code>compression.type</code>	snappy	~60% smaller payloads at low CPU cost.

ServiceRegistry — the in-memory directory

A thread-safe `ConcurrentHashMap` remembering each service and when it was last seen. Health is derived from that timestamp: seen in last 30s = *healthy*, 30–120s = *degraded*, >120s = *unknown*. It also keeps the `SubscribeAlerts` subscribers so alerts can be pushed to gRPC clients.

Interview corner: The Collector keeps stats counters (`batchesReceived` , `metricsDropped` , `kafkaErrors`) as `AtomicLong` and logs them every 30s via `@Scheduled` . Atomic types are used because many gRPC threads update them concurrently.

6. The Message Bus — Apache Kafka & Its Topics

Kafka is the spine of the system. Think of it as a set of durable, append-only logs called **topics**. Producers append messages; consumers read them at their own pace. A topic is split into **partitions** for parallelism.

The four topics (created by the `kafka-init` container)

Topic	Partitions	Retention	Who writes	Who reads
<code>telemetry.metrics</code>	12	7 days	Telemetry Collector	Processing Service
<code>telemetry.processed</code>	12	default	(reserved)	(reserved)
<code>anomalies.detected</code>	4	30 days	AI Service	Alert Service <i>and</i> Backend API
<code>alerts.triggered</code>	4	default	(reserved)	(reserved)

The "one topic, two consumers" trick: Both the Alert Service and the Backend API read `anomalies.detected`, but each uses a *different consumer group* (`alert-service-group` vs `backend-api-group`). In Kafka, each consumer group gets its own copy of every message. So the same anomaly triggers an alert AND gets saved for the dashboard — independently. If you wanted them to share the load instead, you'd put them in the *same* group.

Partitions = parallelism + ordering

`telemetry.metrics` (12 partitions)

part 0	part 1	part 2	...	part 11
--------	--------	--------	-----	---------

▲ ▲ ▲

`hash(serviceId)` decides which partition → same service always same partition (ordered)

Processing Service runs 3 consumer threads x 2 replicas = up to 6 consumers,
each owning some partitions → parallel processing, no two consumers touch the same partition.

Why 12 partitions for metrics but only 4 for anomalies? Metrics are high-volume (every service, every second), so they need more partitions for throughput. Anomalies are rare, so 4 is plenty.

Kafka depends on **Zookeeper** here (Confluent 7.5 image) for cluster coordination. `KAFKA_AUTO_CREATE_TOPICS_ENABLE=false` forces explicit topic creation, which is good practice — it prevents typos from silently creating junk topics.

7. Service Deep-Dive #2 — Processing Service

Java 21 · Kafka consumer · Spring Data JPA · Redis · WebClient Runs as **2 replicas** for throughput.

Job: The workhorse. Take raw metric batches off Kafka, clean them, persist them, cache them, and trigger AI detection. This is where raw data becomes structured, queryable data.

The consumer — at-least-once delivery

```
@KafkaListener(topics="telemetry.metrics", groupId="processing-group", concurrency="3")
public void consume(ConsumerRecord<String,String> record, Acknowledgment ack) {
    try {
        MetricBatchPayload batch = om.readValue(record.value(), MetricBatchPayload.class);
        processor.process(batch);
        ack.acknowledge();           // commit offset ONLY after success
    } catch (Exception e) {
        ack.nack(Duration.ofMillis(1000)); // don't commit -> Kafka redelivers after 1s
    }
}
```

Why this matters (manual acknowledgment): The offset (Kafka's bookmark) is committed *only after* the data is safely in PostgreSQL and Redis. If the service crashes mid-processing, the offset wasn't committed, so on restart Kafka re-delivers the message. Nothing is lost. This is the **at-least-once delivery** guarantee. (Combined with the producer's idempotence, duplicates are minimized.)

The 5-step processing pipeline (`MetricProcessor.process`)

1. **Upsert the service** — find the service by name; create the row if it's the first time we see it, otherwise just bump its `last_seen`.
2. **Validate each metric** against a whitelist `METRIC_SCHEMA`. Unknown metric names are *dropped*; known ones are *clamped* to a valid range (e.g. `cpu_usage` forced into 0–100).
3. **Bulk insert** all valid metrics in one DB round-trip via `saveAll(...)` (`jdbc.batch_size=50`) — far faster than inserting one row at a time.
4. **Update Redis** — store the latest value (a HASH) and append to a sliding window (a LIST trimmed to the last 60 values).
5. **Call the AI** — fire an *async*, fire-and-forget HTTP POST to the Python service with the per-metric windows.

```
// Step 2: validate + clamp
double[] range = METRIC_SCHEMA.get(m.getName());           // e.g. {0.0, 100.0}
double clamped = Math.max(range[0],
    range[1]==Double.MAX_VALUE ? m.getValue() : Math.min(range[1], m.getValue()));

// Step 3: one bulk insert for the whole batch
metricRepo.saveAll(dbRecords);

// Step 5: async, non-blocking AI call
@Async
public void callAnomalyDetector(...) {
    webClientBuilder.build().post().uri(anomalyServiceUrl + "/detect")
        .bodyValue(payload).retrieve().bodyToMono(Map.class)
        .subscribe(resp -> {...}, err -> log.error(...)); // doesn't block the Kafka thread
}
```

Why the AI call is async & "fire-and-forget": If the Processing Service waited for the AI to respond, a slow AI would back up the whole Kafka pipeline. Instead it sends the request on a separate thread pool (`taskExecutor` , core=4/max=16) and moves on. The AI result isn't even needed here — the AI publishes its findings to Kafka itself.

Interview corner — why clamp instead of reject? A CPU reading of 105% is probably a measurement glitch, not garbage. Clamping to 100 keeps the data usable for trend analysis while protecting downstream math from impossible values. Truly unknown metric *names* are dropped because we have no schema/range for them.

8. The Data Stores — PostgreSQL & Redis

PostgreSQL — the permanent record

Defined in `infra/init.sql`, which Postgres runs automatically on first startup. Main tables:

Table	Holds	Notable design
<code>services</code>	One row per monitored service	Seeded with 4 example services.
<code>metrics</code>	Every metric data point (time-series)	Partitioned by day — see below.
<code>anomalies</code>	Every AI-detected anomaly	Has <code>severity</code> CHECK constraint, <code>resolved</code> flag, partial index on open ones.
<code>alerts</code>	Audit log of every alert sent	Records which channels were notified.
<code>metric_baselines</code>	Pre-computed normal stats per metric	mean/std/p50/p95/p99.

Time partitioning — the clever part

The `metrics` table is declared `PARTITION BY RANGE (timestamp)`, and the init script creates one child table *per day* (`metrics_2026_06_29`, etc.). When you insert, Postgres routes the row to the correct day's partition automatically.

```
CREATE TABLE metrics (  
  id BIGSERIAL, service_id VARCHAR(36), metric_name VARCHAR(100),  
  value DOUBLE PRECISION, ... , timestamp TIMESTAMPTZ,  
  PRIMARY KEY (id, timestamp)  
) PARTITION BY RANGE (timestamp);  
  
-- and a maintenance function to delete old data cheaply:  
SELECT drop_old_metric_partitions(7); -- drops partitions older than 7 days
```

Why partition? Deleting 100 million old rows with `DELETE` is slow and bloats the table. **Dropping a whole partition table is instant.** Queries that filter by time also only scan the relevant day(s) — this is called "partition pruning". This is the standard way to handle time-series at scale.

Two handy **views** are also defined: `v_active_anomalies` (unresolved anomalies joined with service names) and `v_service_health` (live health status computed from `last_seen`).

Redis — the speed layer

Redis holds three short-lived data structures (configured with `maxmemory 512mb` and `allkeys-lru` eviction, plus AOF persistence):

Key pattern	Type	TTL	Purpose
metrics:{svc}:latest	HASH	60s	Newest value per metric — for the dashboard "current value".
metrics:{svc}:{metric}:window	LIST	300s	Sliding window of last 60 values — this is what the AI reads.
alerts:dedup:{svc}:{metric}	STRING	300s	Anti-spam lock for alerts (see Alert Service).

```
// sliding window: push, trim to last N, return current contents
redis.opsForList().rightPush(key, value);
redis.opsForList().trim(key, -maxSize, -1); // keep only the last 60
redis.expire(key, WINDOW_TTL);
```

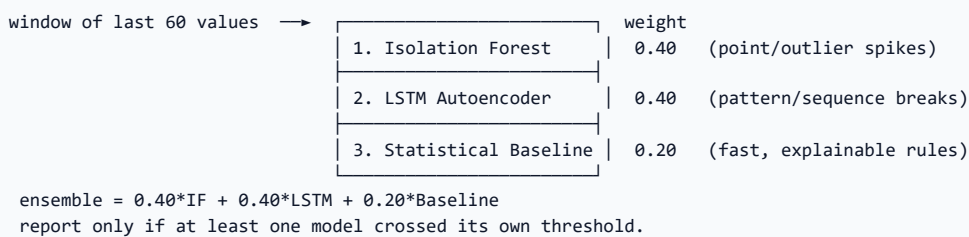
9. Service Deep-Dive #3 — AI Anomaly Detection (the brain)

Python 3.11 · FastAPI · PyTorch · scikit-learn · NumPy · confluent-kafka

Job: Given a recent window of values for a metric, decide how abnormal the latest value is (a score from 0 to 1), how serious it is, and — for serious cases — why it might be happening. Then publish findings to Kafka.

The ensemble idea — three models vote

Instead of trusting one model, the platform runs **three different detectors** and blends their scores. Each is good at catching a different kind of problem.



Model 1 — Isolation Forest (scikit-learn)

Intuition: It builds random decision trees that try to "isolate" each point. Normal points are surrounded by similar points and take many splits to isolate; weird points get isolated quickly. Short isolation path = anomaly.

Clever bit — feature engineering: Raw numbers are weak input. The code converts each window into **8 engineered features** (global z-score, local z-score, rate of change, IQR score, percentile rank, trend slope, coefficient of variation, deviation from median). This makes one model sensitive to spikes, drifts, and pattern breaks at once.

```
return [zscore, local_z, roc, iqr_score, pctlile, slope, cv, dev_median] # 8 features
# It self-trains: buffers samples, trains after 50, retrains every 200.
# Before a model exists, it falls back to a simple z-score.
```

Model 2 — LSTM Autoencoder (PyTorch)

Intuition: An autoencoder is trained to *reconstruct* normal sequences (encode → squeeze through a bottleneck → decode). It becomes good at rebuilding normal patterns. When you feed it an abnormal sequence, it reconstructs it badly → the **reconstruction error (MSE)** is high → anomaly. The LSTM part means it understands *time order*, catching anomalies that look fine point-by-point but are wrong as a sequence (e.g. weird oscillations).

```
class LSTMAutoencoder(nn.Module):    # Encoder LSTM -> bottleneck -> Decoder LSTM
    def forward(self, x):
        _, (hidden, _) = self.encoder(x)
        context = relu(bottleneck_up(relu(bottleneck_down(hidden[-1]))))
        repeated = context.repeat(seq_len)
        return output_layer(self.decoder(repeated))
# threshold = 95th percentile of training errors; trains in a BACKGROUND thread
# so it never blocks live scoring. Falls back to z-score until trained.
```

Model 3 — Statistical Baseline

Intuition: Fast, always-available, and *explainable*. Four classic rules, each giving a 0–1 sub-score; the final score is the max:

1. **Z-score** — how many standard deviations from the mean?
2. **IQR fence** — is it outside $Q1 - 1.5 \cdot IQR \dots Q3 + 1.5 \cdot IQR$?
3. **Rate of change** — is the last jump unusually big vs typical volatility?
4. **Sustained trend** — are the last 5–6 points all rising or all falling?

Putting it together (the /detect endpoint)

```
if_score, lstm_score, base_result = await asyncio.gather( # run all 3 in parallel
    asyncio.to_thread(iso_forest.score, window, metric_name),
    asyncio.to_thread(lstm_model.score, window, metric_name),
    asyncio.to_thread(baseline.analyze, window))
ensemble = 0.40*if_score + 0.40*lstm_score + 0.20*base_result["score"]
severity = critical(>=.90) / high(>=.75) / medium(>=.55) / low
if ensemble >= 0.75: root_cause = rca.analyze(...) # only for serious ones
# publish {service, anomalies[...] } to Kafka topic "anomalies.detected"
```

Root Cause Analyzer

A hand-built **causal knowledge graph**: for each metric, a list of likely culprits with human explanations. E.g. for `latency_p99` it suggests checking `cpu_usage`, `db_connections`, `gc_pause_ms`, `queue_depth`. It also adds concrete recommended actions ("check DB slow query log for queries > 1s"). This turns a number into something an engineer can act on.

Interview corner — "is the RCA real AI?" Be honest: the detection (Isolation Forest + LSTM) is genuine ML; the RCA is a *rule-based expert system* (encoded domain knowledge), not a learned model. Real tools like Datadog build these graphs from historical incident data — this is a simplified, transparent version.

10. Service Deep-Dive #4 — Alert Service

Java 21 · Kafka consumer · Redis · Slack webhook · JavaMail (SMTP)

Job: Consume detected anomalies and notify humans — without spamming them.

The routing pipeline (`AlertRouter.route` — runs `@Async`)

1. **Severity filter:** drop anything below `MIN_ALERT_SEVERITY` (default *medium*). So *low* anomalies are recorded but don't page anyone.
2. **Deduplication:** if the same service+metric alerted in the last 300s, suppress it.
3. **Format** a readable message (with emoji per severity, values, and root cause).
4. **Persist** to the `alerts` table (audit trail).
5. **Notify:** always Slack; additionally **email** if severity is *high* or *critical*.

Deduplication — the anti-storm lock

```
public boolean isDuplicate(String serviceId, String metricName) {
    String key = "alerts:dedup:" + serviceId + ":" + metricName;
    Boolean isNew = redis.opsForValue().setIfAbsent(key, "1", Duration.ofSeconds(ttl));
    return Boolean.FALSE.equals(isNew);    // false = key already existed = duplicate
}
```

Why this is elegant: Redis `SETNX` (set-if-not-exists) is *atomic*. The first alert sets the key and returns "new"; every repeat within 300s finds the key and is suppressed. After 300s the key expires and alerts can fire again. Without this, a CPU stuck at 95% for 10 minutes would generate hundreds of identical Slack messages.

The two notifiers

- **SlackAlerter** — POSTs to a Slack Incoming Webhook using the `attachments` API, with a colored sidebar per severity (red/orange/yellow/green). Silently skips if no webhook is configured.
- **EmailAlerter** — builds an HTML email via Spring's `JavaMailSender` and sends through SMTP (Gmail by default). Skips silently if SMTP isn't configured.

Both notifiers are **optional and fail-safe**: if credentials are missing they just `log.debug` and return, so the platform runs fine out-of-the-box without any Slack/email setup.

11. Service Deep-Dive #5 — Backend API + WebSocket

Java 21 · Spring Web (REST) · Spring WebSocket · JPA · Redis · Kafka consumer

Job: The bridge between stored data and the browser. Two halves: a **REST API** for on-demand queries, and a **WebSocket** for live push.

REST endpoints

Endpoint	Returns
GET /api/v1/services	All services + computed health status.
GET /api/v1/metrics/{svc}?metricName=&hours=	Time-series from PostgreSQL.
GET /api/v1/metrics/{svc}/latest	Latest values straight from Redis (no DB hit).
GET /api/v1/anomalies?severity=&resolved=&hours=	Filtered anomaly list (one flexible JPA query).
PATCH /api/v1/anomalies/{id}/resolve	Mark an anomaly resolved (the dashboard "Resolve" button).
GET /api/v1/anomalies/stats	Header-card counts (total / active / critical / high).

The flexible filter query is a nice trick — one query handles all optional filters:

```
WHERE (:serviceId IS NULL OR a.serviceId = :serviceId)
  AND (:severity IS NULL OR a.severity = :severity)
  AND (:resolved IS NULL OR a.resolved = :resolved)
  AND a.detectedAt >= :since
ORDER BY a.detectedAt DESC
```

The live WebSocket (DashboardWebSocketHandler)

Browsers connect to `ws://localhost:8000/ws/dashboard`. The handler keeps all open sessions in a `ConcurrentHashMap`. A `@Scheduled(fixedRate=1000)` job runs **every second**, builds a snapshot (services + active anomalies + stats), serializes to JSON, and broadcasts to every open session. Dead sessions are pruned during the broadcast.

```
@Scheduled(fixedRate = 1000)
public void broadcast() {
    String json = om.writeValueAsString(buildSnapshot());
    sessions.forEach((id, sess) -> {
        if (sess.isOpen()) sess.sendMessage(new TextMessage(json));
        else dead.add(id);           // clean up closed connections
    });
}
```

Why push instead of poll? One server push per second to all clients is far cheaper than every browser polling the REST API every second. The server controls the cadence and the data stays fresh.

Also: AnomalyKafkaListener

Separately, the Backend API consumes `anomalies.detected` (group `backend-api-group`) and saves each anomaly to PostgreSQL via `AnomalyService.save(...)` — so the REST history endpoints have data to return. This is the second consumer of that topic (the first being the Alert Service).

12. Service Deep-Dive #6 — React Dashboard

React 18 · Recharts · WebSocket client · served by nginx

Job: Show everything to a human, live. Three tabs: *Dashboard* (stat cards + live metric charts + anomaly feed), *Services* (health badges), and *Anomalies* (full list).

How it stays live

On mount, it opens a WebSocket to the Backend API. Every snapshot updates React state, which re-renders the charts. It also **auto-reconnects** after 3 seconds if the connection drops:

```
ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  if (data.type !== 'snapshot') return;
  setServices(data.services);
  setAnomalies(data.active_anomalies);
  setStats(data.stats);
  // keep a rolling 60-point history per metric for the sparkline charts
};
ws.onclose = () => { setConnected(false); setTimeout(connect, 3000); }; // reconnect
```

Nice UX touches

- **Demo data fallback:** if no real data has arrived yet, it injects fake services/charts so the dashboard never looks broken on first load.
- **Connection indicator:** a green/red dot + "Live / Connecting..." in the header.
- **Resolve button:** calls `PATCH /anomalies/{id}/resolve` and optimistically removes the row.
- **nginx** serves the built static files and proxies `/api/` and `/ws/` to the Backend API, with the special headers a WebSocket upgrade needs.

Interview corner: The whole UI is one `App.jsx` using only React hooks (`useState` , `useEffect` , `useRef` , `useCallback`) — no Redux. State is simple enough that a single component tree driven by WebSocket snapshots is sufficient.

13. Two Complete End-to-End Walkthroughs

Walkthrough A — A normal metric (the happy path)

1. test_sender.py sends cpu_usage=42 (normal) over gRPC.
2. Collector validates, publishes JSON to telemetry.metrics (key=serviceId).
3. Processing Service consumes, clamps (42 is fine), bulk-inserts to Postgres, pushes 42 onto the Redis window, commits the Kafka offset.
4. Async POST to AI /detect with the cpu_usage window [..., 41, 43, 42].
5. All 3 models score it LOW (it blends in). No model crosses threshold.
6. AI returns {anomalies: []}. Nothing published. No alert. No DB anomaly row.
7. Backend API's 1s WebSocket snapshot still updates the live chart line. Dashboard shows cpu_usage ticking along, service = "healthy". Done.

Walkthrough B — An anomaly + alert (the interesting path)

1. SPIKE=1 python test_sender.py -> sends cpu_usage~95, latency_p99~800, error_rate~10.
2. Collector -> Kafka (same as before).
3. Processing Service stores them, window now ends in [...,43,42,95].
4. Async POST to AI /detect.
5. AI scores the cpu_usage window:
 - Isolation Forest: 95 is far from the cluster -> if_score = 0.88
 - LSTM Autoencoder: can't reconstruct the spike -> lstm_score = 0.83
 - Statistical: z-score huge, IQR breached -> base_score = 0.91
 - ensemble = .40*.88 + .40*.83 + .20*.91 = ~0.866 -> severity = HIGH
6. ensemble (0.866) >= 0.75 -> RootCauseAnalyzer runs:
 - "Check cpu_usage causes: request_rate spike, thread storms, GC, queue backlog..."
7. AI publishes the anomaly to anomalies.detected.
- 8a. ALERT SERVICE consumes it:
 - HIGH >= medium (passes filter) -> not a duplicate -> save to alerts table
 - > Slack message sent (orange sidebar) -> email also sent (HIGH).
- 8b. BACKEND API consumes the SAME message (different group):
 - saves the anomaly row to Postgres.
9. Next 1s WebSocket snapshot includes this anomaly.
10. Dashboard: "Active Anomalies" card increments, the Anomaly Feed shows a HIGH cpu_usage row on payment-service with the root-cause text. Engineer clicks Resolve.

Why ~30–60s before detection? Each model needs enough recent points to judge "normal" (Isolation Forest needs 50 samples to train; LSTM needs 100; even the fallback needs ≥ 5). The window fills at ~1 value/second, so the first confident detections come after the window has enough history.

14. Cross-Cutting Design Concepts (the "why it's good")

These are the ideas an interviewer loves. Each appears in multiple services.

1. Loose coupling via Kafka

Services never block on each other in the hot path. Producer speed is independent of consumer speed. A down consumer just means a backlog that gets processed on recovery — no data loss, no cascading failure.

2. At-least-once delivery + idempotent producer

Consumers commit offsets only after success (`ack.acknowledge()` after DB write). The producer uses `enable.idempotence=true` and `acks=all` . Together: no lost messages, minimal duplicates.

3. Two-tier storage (hot vs cold)

Redis = "hot" data (last 60s/300s, in-memory, sub-ms) for live reads and the AI window. PostgreSQL = "cold" durable history for queries and audit. Each tool does what it's best at.

4. Time partitioning

Daily partitions make inserts fast, time-range queries fast (pruning), and old-data deletion instant (drop partition vs slow DELETE).

5. Ensemble ML + thresholded reporting

Three diverse models reduce false positives/negatives — a true anomaly usually trips more than one. Weighted blend + per-model thresholds make it tunable via environment variables.

6. Async everywhere it matters

The AI HTTP call (Processing), alert delivery (Alert Service), and LSTM training (AI) all run on separate threads so the critical path is never blocked.

7. Backpressure & flow control

gRPC streaming + Kafka buffering + consumer `max.poll.records=500` mean bursts are absorbed gracefully instead of overwhelming the database.

8. Graceful degradation

No Slack/email config → alerts skip silently. AI not trained yet → z-score fallback. AI down → metrics still stored. Dashboard with no data → demo data. The system always does *something* sensible.

9. Horizontal scalability

Processing runs 2 replicas; Kafka's 12 partitions allow up to 12 parallel consumers. Add replicas to scale throughput — the partition count is the ceiling.

10. Containerized, reproducible, health-checked

Docker Compose with health checks and `depends_on: condition: service_healthy` ensures services start in the right order (DB/Kafka ready before apps). Multi-stage Dockerfiles keep images small and run as a non-root user.

15. Interview Q&A — Anticipated Questions & Strong Answers

Architecture & design

Q: Walk me through what happens when a metric is sent.

A: A service streams a `MetricBatch` over gRPC to the Collector → Collector validates and publishes JSON to Kafka (`telemetry.metrics` , keyed by `serviceld`) → Processing Service consumes, validates/clamps, bulk-inserts to Postgres, updates Redis (latest + 60-value window), and fires an async POST to the AI → AI runs 3 models, computes an ensemble score, and if anomalous publishes to `anomalies.detected` → Alert Service sends Slack/email and Backend API saves it + pushes to the dashboard via WebSocket.

Q: Why use Kafka instead of having services call each other directly?

A: Decoupling, durability, and buffering. Producers and consumers run at independent speeds; bursts are absorbed; data survives consumer downtime (7–30 day retention); and multiple consumer groups can read the same stream independently (Alert Service + Backend API both read anomalies). Direct calls would create tight coupling and cascading failures.

Q: Why is the AI in Python but everything else in Java?

A: ML libraries (PyTorch, scikit-learn) are Python-native and far more mature there. Doing ML in Java would be painful. The design uses the best language per job and connects them with neutral protocols (HTTP/JSON for the call, Kafka for the result). Java handles robust, concurrent data plumbing; Python handles the math. Clean language boundary.

Q: Why gRPC for ingestion but REST for the dashboard API?

A: Ingestion is high-volume and machine-to-machine — gRPC's binary Protobuf encoding and persistent streaming connection are much more efficient than REST. The dashboard API is low-volume, human-facing, and benefits from REST's simplicity and browser-friendliness (plus WebSocket for live push).

Reliability & correctness

Q: How do you guarantee no metric is lost?

A: Producer side: `acks=all` + retries + idempotence. Consumer side: manual offset commit — `ack.acknowledge()` is called only *after* the data is persisted to Postgres and Redis. If processing fails or the service crashes, the offset isn't committed, so Kafka re-delivers. That's at-least-once delivery.

Q: At-least-once means possible duplicates. How do you handle them?

A: The idempotent producer prevents duplicates from *producer* retries. For the rare consumer-replay duplicate, the impact is minor (a duplicate metric row); and alert spam from repeated anomalies is killed by the Redis SETNX dedup lock. If strict exactly-once metric storage were required, I'd add a unique constraint or upsert keyed on (`service_id`, `metric_name`, `timestamp`).

Q: How does message ordering work?

A: Kafka guarantees order *within a partition*. Using `serviceld` as the message key sends all of one service's metrics to the same partition, so they're processed in order. Different services can be processed in parallel on different partitions.

Q: What happens if the AI service is down?

A: Nothing breaks. The Processing Service's call is async fire-and-forget with an error callback — metrics are still stored. No anomalies are detected during the outage, but when the AI recovers it resumes scoring new windows. The pipeline never blocks on the AI.

The AI / ML

Q: Explain the three models and why an ensemble.

A: Isolation Forest catches point outliers (sudden spikes) using 8 engineered features. The LSTM Autoencoder catches sequence/pattern anomalies via reconstruction error (it understands time order). The Statistical Baseline is fast, always available, and explainable (z-score, IQR, rate-of-change, trend). Ensembling (0.4/0.4/0.2) reduces false alarms — a real anomaly usually trips more than one, and each covers the others' blind spots.

Q: How does the LSTM Autoencoder actually detect anomalies?

A: It's trained only on normal data to reconstruct input sequences. It gets good at rebuilding normal patterns. For an abnormal sequence it reconstructs poorly, so the MSE between input and output is high. The anomaly threshold is the 95th percentile of training-set errors; current error ÷ threshold gives the score. Training runs in a background thread so it never blocks live scoring.

Q: How do the models train without a labeled dataset?

A: They're *unsupervised*. Isolation Forest assumes ~5% contamination and learns the structure of incoming data; the LSTM learns to reconstruct whatever it has seen (assumed mostly normal). Both buffer live samples and self-train (IF after 50 samples, retrain every 200; LSTM after 100, retrain every 300). Before training completes, both fall back to a simple z-score so the system works immediately.

Q: Is the Root Cause Analysis machine learning?

A: No — it's a rule-based expert system: a hand-built causal graph mapping each metric to likely related metrics with explanations and recommended actions. The *detection* is real ML; the RCA is encoded domain knowledge. Production tools learn these graphs from incident history; this is a transparent, simplified version. I'd be upfront about that distinction.

Q: How would you reduce false positives?

A: Tune the per-model thresholds (they're env vars), increase the ensemble report threshold, require ≥2 models to fire, lengthen the window, add seasonality awareness (time-of-day baselines), and feed analyst "resolve/false-positive" feedback back into thresholds. The `metric_baselines` table is already set up to hold learned normal stats.

Data & storage

Q: Why partition the metrics table by day?

A: Time-series grows forever. Daily partitions make inserts target a small table, let time-range queries prune to only relevant days, and make deleting old data instant (DROP partition vs a slow, bloating DELETE). The `drop_old_metric_partitions(7)` function automates retention.

Q: Why both Redis and PostgreSQL?

A: Different jobs. Redis is in-memory and sub-millisecond — perfect for the AI's hot sliding window, "latest value" dashboard reads, and atomic dedup locks, all short-lived (TTLs). Postgres is durable and SQL-queryable — perfect for history, audit, and complex filtered queries. Using Redis as the AI window avoids hammering Postgres for recent data on every detection.

Q: Why bulk insert instead of one insert per metric?

A: A batch has ~9 metrics; inserting them one-by-one is ~9 network round-trips. `saveAll` with Hibernate `batch_size=50` sends them in one round-trip — dramatically higher throughput, especially at scale with 2 replicas × 3 threads consuming continuously.

Alerting & scaling

Q: How do you prevent alert spam?

A: A Redis SETNX dedup lock keyed on service+metric with a 300s TTL. The first alert sets the key; repeats within the window find it and are suppressed; after it expires, alerting resumes. There's also a severity filter so low-severity anomalies never page anyone. Email is reserved for high/critical.

Q: How would you scale this to 10× the traffic?

A: Add Processing replicas up to the partition count (12), then increase partitions for more parallel consumers. Scale the Collector horizontally behind a gRPC load balancer. Run more AI workers (`ANOMALY_SERVICE_WORKERS`) or shard by metric. For Postgres, the partitioning already helps; beyond that, a dedicated time-series DB (TimescaleDB) or read replicas. Kafka itself scales by adding brokers.

Q: What's the single biggest bottleneck?

A: Most likely PostgreSQL write throughput under very high metric volume, since every batch is persisted. Mitigations: more partitions, batched/async writes, TimescaleDB, or downsampling old data. The AI is the next candidate, but it's already parallelized and off the hot path.

"Gotcha" / honesty questions

Q: What are the weaknesses or things you'd improve?

A: (1) ML models train per-process in memory — with multiple AI workers each trains separately; a shared model store would be better. (2) No authentication on the APIs/gRPC. (3) `telemetry.processed` and `alerts.triggered` topics are reserved but unused. (4) `CrossOrigin("*")` is open for demo convenience. (5) Exactly-once metric storage isn't enforced. (6) RCA is rules, not learned. Naming these shows I understand the tradeoffs that were made for a demo/portfolio build.

Q: How is configuration managed?

A: Environment variables, defaulted in each service's `application.yml` and overridden by `docker-compose.yml` (with a shared `x-java-env` YAML anchor) and the `.env` file for secrets like the Slack webhook and SMTP password. This keeps secrets out of code and makes the same image run in any environment.

Q: How do services start in the right order?

A: Docker Compose health checks plus `depends_on: condition: service_healthy`. Apps wait for Postgres/Redis/Kafka to report healthy. A one-shot `kafka-init` container creates the topics before the apps need them.

16. Glossary & Quick Reference

Key terms

Term	Plain meaning
Telemetry / Metric	A number a service reports about itself (CPU, latency, errors...).
Observability	Being able to understand a system's internal state from its outputs.
gRPC	A fast binary RPC framework using Protocol Buffers; supports streaming.
Kafka topic / partition	A durable log of messages; split into partitions for parallelism + ordering.
Consumer group	A set of consumers sharing a topic's load; each group gets its own copy of all messages.
Offset	Kafka's bookmark — how far a consumer group has read in a partition.
Anomaly	A metric value the AI judges abnormal vs learned normal.
Ensemble	Combining several models' scores into one decision.
Autoencoder	A neural net that learns to reconstruct its input; high reconstruction error = anomaly.
Isolation Forest	A tree-based model that isolates outliers quickly.
Idempotence	Doing the same operation twice has the same effect as once (no duplicates).
Upsert	Insert if new, otherwise update the existing row.
TTL	Time-to-live — how long a cache key lives before auto-expiry.
Backpressure	Slowing/buffering producers so consumers aren't overwhelmed.

Run it yourself (commands)

```
# start everything
cp .env.example .env           # optionally add Slack/Gmail creds
docker compose --env-file .env up --build -d
docker compose ps              # all should be Up/healthy

# send test data
pip install grpcio grpcio-tools
cd telemetry-collector/src/main/proto
python -m grpc_tools.protoc -I. --python_out=../../../../ --grpc_python_out=../../../../ telemetry.proto
cd ../../../../../../
python test_sender.py           # normal metrics
SPIKE=1 python test_sender.py  # trigger an anomaly

# open
http://localhost:3000           # dashboard
http://localhost:8001/docs      # AI API docs
http://localhost:8000/api/v1/services # REST
```

The one-paragraph elevator pitch (memorize this)

"It's an end-to-end, AI-powered observability platform built as polyglot microservices. Services stream metrics over gRPC to a Java collector, which publishes to Kafka. A Java processing service stores them in PostgreSQL and Redis and asks a Python AI service to check for anomalies using an ensemble of three models — Isolation Forest, an LSTM autoencoder, and statistical rules. Detected anomalies flow back through Kafka to an alert service (Slack/email, with Redis-based deduplication) and to a backend API that persists them and pushes live updates to a React dashboard over WebSocket. Everything is decoupled through Kafka for resilience, uses at-least-once delivery for correctness, and runs with one `docker compose up`."